STRUCTURES DE DONNÉES

1. Tableaux (arrays) :

1. tableau statique :

- **Définition :**

Un tableau statique est une collection de donnees du meme type avec une taille fixe determinee lors de sa creation.

- **Explication :**

On connat le nombre d'elements a l'avance. Les indices commencent à 0 .

Exemple :

int[] nombres = new int[5]; // tableau de 5 entiers

nombres[0] = 10;

nombres[1] = 20;

nombres[2] = 30;

nombres[3] = 40;

nombres[4] = 50;

```
for (int i = 0; i < nombres.Length; i++)
{
   Console.WriteLine("Élément " + i + " : " + nombres[i]);
}
```

2. tableau dynamique :

- Définition :

Un tableau dynamique peut changer de taille pendant l'execution.

Exemple :

```
int[] nombres = { 1, 2, 3 };
Array.Resize(ref nombres, 5); // agrandit le tableau à 5 éléments
nombres[3] = 4;
nombres[4] = 5;
foreach (int n in nombres){
   Console.WriteLine(n);
}
```

2. Listes (List<T>):

- Définition :

List est une collection dynamique qui peut grandir ou rétrécir selon les besoins.

- T est le type des elements (int, string…).

- Explication :
  - Plus flexible que les tableaux.
  - Permet d'ajouter (Add), supprimer (Remove) ou inserer (Insert) des elements facilement.

Exemple :

```
using System;
using System.Collections.Generic;
class Program{
   static void Main(){
      List<string> fruits = new List<string>();
      fruits.Add("Pomme");
      fruits.Add("Banane");
      fruits.Add("Cerise");
      fruits.Remove("Banane"); // supprime Banane
      fruits.Insert(1, "Orange"); // ajoute Orange à l'indice 1
      foreach (string fruit in fruits){
         Console.WriteLine(fruit);
      }
   }
}
```

3. Dictionnaires (Dictionary <Tkey, TValue>):

- Définition :

Un dictionnaire est une collection clé-valeur, ou chaque cle unique correspond a une valeur. TKey : type de la cle TValue : type de la valeur

- Explication :

Permet un acces rapide aux valeurs via leur cle. Utile pour stocker des donnees associatives (ex : nom → age).

Exemple :

```
using System;
using System.Collections.Generic;
class Program{
  static void Main(){
    Dictionary<string, int> ages = new Dictionary<string, int>();
    ages["Alice"] = 25;
    ages["Bob"] = 30;
    ages["Charlie"] = 22;
    // Accéder à une valeur via sa clé
    Console.WriteLine("Âge de Bob : " + ages["Bob"]);
    // Parcourir le dictionnaire
```

```
    foreach (KeyValuePair<string, int> entry in ages){
        Console.WriteLine(entry.Key + " a " + entry.Value + " ans.");
    }
  }
}
```

4. Matrices :

- Définition :

Une matrice est un tableau a deux dimensions qui permet de stocker des donnees sous forme de lignes et de colonnes.

- Syntaxe générale :

```
type[,] nomMatrice = new type[nbLignes, nbColonnes];
```

- Déclaration et initialisation :

```
using System;
class Program{
  static void Main(){
    int[,] matrice = new int[2, 3]; // 2 lignes, 3 colonnes
    // Affecter des valeurs
    matrice[0, 0] = 1;
    matrice[0, 1] = 2;
    matrice[0, 2] = 3;
```

```
        matrice[1, 0] = 4;
        matrice[1, 1] = 5;
        matrice[1, 2] = 6;
        // Affichage
        for (int i = 0; i < 2; i++){
            for (int j = 0; j < 3; j++){
                Console.Write(matrice[i, j] + " ");
            }
            Console.WriteLine();
        }
    }
}
```

- Initialisation directe :

```
int[,] matrice = { { 1, 2, 3 }, { 4, 5, 6 } };
for (int i = 0; i < matrice.GetLength(0); i++) {
    for (int j = 0; j < matrice.GetLength(1); j++) {
        Console.Write(matrice[i, j] + " ");
    }
    Console.WriteLine();
}
```

- Matrice de chaînes:

```
string[,] tableauNoms = { {"Alice", "Bob"}, {"Charlie", "Diana"} };
for (int i = 0; i < tableauNoms.GetLength(0); i++){
    for (int j = 0; j < tableauNoms.GetLength(1); j++){
        Console.Write(tableauNoms[i, j] + " ");
    }
    Console.WriteLine();
}
```